

TEAM VELTRIX

Product Requirements Document

VERA

Verification Engine for Risk Analysis

An AI-powered financial trust verification platform that detects fraud rings and scores entity trustworthiness using network analysis. Built on the proven GRACE architecture, upgraded for the Squad Hackathon 3.0 challenge: Proof of Life.

Hackathon	Squad Hackathon 3.0 — Proof of Life (Challenge 01)
Team	Team Veltrix
Members	Team Veltrix (4 members)
Built On	GRACE — Graph-based Risk And Compliance Engine (existing production system)
Live Demo	https://grace-drab.vercel.app (GRACE baseline)
Repo	https://github.com/Techdee1/GRACE
Theme	Smart Systems: The Intelligent Economy
Challenge	Build an AI verification system that scores trustworthiness where fraud is common

01

From GRACE to VERA

VERA is not a new product built from scratch. It is a strategic upgrade of GRACE, a production-deployed AML compliance platform built by Team Veltrix and placed 3rd at the Antler Lagos x Lua AI Hackathon, evaluated by an AI scoring engine across 13 dimensions.

For Squad Hackathon 3.0, GRACE has been reframed, extended, and deepened to directly address Challenge 01: Proof of Life. The core architecture is proven and live. What changes is the narrative, the Squad API integration, the ML depth, and three new features designed specifically to maximize the judging criteria.

WHY VERA WINS THIS HACKATHON

Most teams will build from scratch in 48 hours. VERA arrives with a production-deployed system, a live demo, graph-based AI, LLM reasoning, an immutable audit trail, and a prior hackathon placement.

The only thing between VERA and first place is Squad API integration and the three new features described in this document. Everything else is already built.

The Reframe

GRACE was pitched as an AML compliance tool for compliance officers. VERA is pitched as a financial trust verification engine. Same product, different story — and the new story maps directly to Challenge 01.

Dimension	GRACE (original)	VERA (Squad Hackathon)
Product description	AML/KYC compliance platform	AI-powered financial trust verification engine
Core output	Suspicious Transaction Report (STR)	Trust score, risk flag, verification verdict
Challenge framing	Regulatory compliance	Proof of Life — fraud detection and trust scoring
Primary user	Compliance officer	Financial institutions, payment processors, regulators
Squad API	Not integrated	Central — data source AND action layer

ML depth	Heuristic + LLM	Heuristic + Isolation Forest + LLM + NLP
Deployment	Live on Vercel + DigitalOcean	Same, with Squad webhook endpoint added

Existing GRACE Architecture

This is what is already built, deployed, and working. VERA inherits all of this. No re-implementation required.

Tech Stack

Layer	Technology	Status
Frontend	React 18 + TypeScript, Tailwind CSS, D3.js, TanStack Query	Live — grace-drab.vercel.app
Backend	FastAPI (Python 3.11), Celery workers via Redis BLPOP, SQLAlchemy, Pydantic v2	Live — DigitalOcean App Platform
Graph DB	Neo4j Community Edition	Live
Relational DB	PostgreSQL 15	Live
Cache / Queue	Redis 7	Live
LLM	Groq API — llama-3.3-70b-versatile	Live
Entity Resolution	RapidFuzz fuzzy name matching	Live
DevOps	Docker Compose (dev), GitHub Actions (CI)	Live

Existing Pipeline

The full detection and reporting pipeline is already implemented and proven on a synthetic dataset of 500 entities and 2,000 transactions.

End-to-end pipeline (already working)

```
POST /api/v1/transactions/ingest
|
v
[ Celery Worker ]
|
+--> Entity Resolution (RapidFuzz fuzzy matching)
|   Exact BVN/NIN match -> fuzzy name fallback
|
+--> Graph Update (Neo4j)
|   Upsert entity nodes + transaction edges
|
```

```
+--> Heuristic Detection
| POS Cash-Out Ring (5+ senders, 72hr, NGN 500k)
| Shell Director Web (3+ businesses, shared directors)
| Layered Transfer Chain (4+ hops, 48hr reconsolidation)
|
+--> Risk Scoring (0.0-1.0)
| Direct entity score + neighbor propagation (50% decay)
|
+--> Alert Creation
| Fingerprint deduplication (SHA-256)
| Redis distributed lock (no concurrent duplicates)
|
+--> STR Generation (Groq LLM)
| NFIU-format draft from graph evidence
|
+--> Audit Log (immutable, payload-hashed)
```

Existing API Endpoints

Method	Endpoint	Description
POST	/api/v1/transactions/ingest	Ingest single or batch transactions (async, returns 202 + job_id)
GET	/api/v1/entities/{id}	Entity profile + 1-hop neighborhood
GET	/api/v1/entities/{id}/risk	Risk score + explanation
GET	/api/v1/alerts	List of open alerts (paginated)
GET	/api/v1/alerts/{id}	Alert detail + subgraph evidence
POST	/api/v1/str/generate	Generate NFIU-format STR draft (Groq)
GET	/api/v1/str/{id}	Fetch STR draft
PATCH	/api/v1/str/{id}/decision	Approve or reject an STR
GET	/api/v1/jobs/{job_id}	Poll async job status
GET	/api/v1/entities	List entities (paginated)
GET	/api/v1/str	List STR drafts

Existing Frontend Features

- Dashboard with live metric cards (500 entities, open alerts, STR count)
- D3.js force-directed graph explorer with risk-based node coloring (red/yellow/green)
- Alert list with pattern type, risk score, and entity count
- Alert detail with subgraph visualization and entity evidence panel
- STR generation from alert detail with optional reviewer notes
- STR detail page with AI-generated narrative, approve/reject workflow
- STR PDF download (jsPDF with NFIU-format sections)
- Entity detail page with 1-hop network, transaction summary, linked alerts
- Landing page with animated network graph hero
- Immutable audit trail logging on every action

Scoring Gap Analysis

Challenge 01 is judged across six criteria. Here is where GRACE currently stands and what VERA needs to close each gap.

Criterion	Weight	GRACE Score	VERA Target	Gap
AI Technical Depth	30%	7/10 — heuristic + LLM only, no trained model	9/10	Add Isolation Forest + NLP entity verification
Squad API Integration	20%	0/10 — not integrated	9/10	Option C: webhook + payment trigger
Problem Relevance & Domain Insight	15%	8/10 — deep Nigerian fraud knowledge	9/10	Add real user quotes + documented case study
Solution Design & Scalability	15%	9/10 — production architecture, deployed	9/10	Add multi-tenancy diagram
Presentation & Demo	15%	8/10 — live demo works	9/10	Trust score badge + before/after moment
Impact Potential (bonus)	10%	5/10 — no responsible AI coverage	8/10	Add Responsible AI panel

EXISTENTIAL RISK

Squad API integration is a disqualification condition. A solution with no meaningful Squad API integration will not be eligible for top placements regardless of scores in other criteria. This is the first thing to build and the last thing to break before the demo.

04

Squad API Integration — Option C

Squad API appears at both ends of the VERA pipeline. It is the data source that feeds the graph and the action layer that closes the verification loop. This is the strongest possible integration story.

THE PITCH LINE

"VERA sits on top of Squad's payment infrastructure and watches every transaction for fraud patterns in real time. When a fraud ring is confirmed, VERA initiates the financial consequence through Squad. Squad powers both the intelligence and the action."

Integration Point 1 — Squad as Data Source

Every Squad transaction fires a webhook event into VERA's backend. VERA parses the payload, maps it to its transaction schema, and feeds it into the existing ingest pipeline. The graph updates in real time.

Flow

```
Squad transaction occurs
|
v
Squad fires POST to https://vera-backend.com/api/v1/webhooks/squad
|
v
[FastAPI webhook endpoint]
- Verify HMAC signature (reject if invalid)
- Parse Squad payload -> TransactionIngestItem
- Call existing ingest pipeline
|
v
[Existing Celery Worker]
- Entity resolution
- Graph update (Neo4j)
- Pattern detection
- Risk scoring
- Alert creation if threshold exceeded
|
v
Dashboard updates in real time
```

New endpoint to build:

backend/app/api/routes/webhooks.py (NEW FILE)

```
from fastapi import APIRouter, Request, HTTPException, Header
from app.services.ingest_service import create_ingest_job
from app.core.config import settings
import hmac, hashlib, json
```



```

router = APIRouter(prefix="/webhooks", tags=["webhooks"])

def verify_squad_signature(payload: bytes, signature: str) -> bool:
    expected = hmac.new(
        settings.squad_webhook_secret.encode(),
        payload,
        hashlib.sha512
    ).hexdigest()
    return hmac.compare_digest(expected, signature)

@router.post("/squad")
async def squad_webhook(
    request: Request,
    x_squad_signature: str = Header(None),
    db: Session = Depends(get_db)
):
    payload = await request.body()

    if not verify_squad_signature(payload, x_squad_signature or ""):
        raise HTTPException(status_code=401, detail="Invalid signature")

    data = json.loads(payload)
    event_type = data.get("Event")

    if event_type == "charge_completed":
        txn = data.get("Body", {})
        ingest_payload = TransactionIngestItem(
            id=txn.get("transaction_ref"),
            sender_id=txn.get("sender_id") or txn.get("customer_identifier"),
            receiver_id=txn.get("merchant_id"),
            amount_ngn=float(txn.get("transaction_amount", 0)) / 100,
            timestamp=txn.get("createdAt"),
            channel="squad_payment",
            sender_name=txn.get("sender_name"),
            receiver_name=txn.get("merchant_name"),
        )
        job = create_ingest_job(db=db, payload=ingest_payload)
        return {"status": "accepted", "job_id": str(job.id)}

    return {"status": "ignored", "event": event_type}

```

Integration Point 2 — Squad as Action Layer

When a compliance officer approves and files an STR, VERA initiates a Squad payment representing the regulatory action. The Squad transaction reference is stored on the STR record and displayed on the detail page.

Flow

```

Compliance officer clicks Approve + File
|
v
POST /api/v1/str/{id}/file
|
v
[Backend]

```

```
- Update STR status to "filed"
- Call Squad POST /payment/initiate
- Store Squad transaction_ref on STR record
- Write to audit log
|
v
STR detail page shows:
"Filed via Squad — TXN#abc123def456"
[Squad logo badge]
```

New endpoint to build:

backend/app/api/routes/str_drafts.py (ADD THIS ENDPOINT)

```
@router.post("/{str_id}/file")
def file_str_draft(
    str_id: str,
    db: Session = Depends(get_db),
):
    draft = db.get(STRDraft, str_id)
    if not draft:
        raise HTTPException(status_code=404, detail="STR not found")
    if draft.decision != "approved":
        raise HTTPException(
            status_code=409,
            detail="STR must be approved before filing"
        )

    # Trigger Squad payment
    squad_response = requests.post(
        "https://sandbox.squadco.com/payment/initiate",
        headers={
            "Authorization": f"Bearer {settings.squad_secret_key}",
            "Content-Type": "application/json",
        },
        json={
            "email": settings.compliance_email,
            "amount": 10000, # NGN 100.00 (in kobo)
            "currency": "NGN",
            "initiate_type": "inline",
            "transaction_ref": f"VERA-STR-{str_id[:8]}",
            "callback_url": f"{settings.base_url}/api/v1/webhooks/squad",
        }
    )

    if squad_response.status_code == 200:
        squad_data = squad_response.json()
        txn_ref = squad_data.get("data", {}).get("transaction_ref")
        draft.squad_transaction_ref = txn_ref
        draft.status = "filed"
        db.commit()

    log_audit_event(db=db, action="str_filed",
        payload_json={"str_id": str_id,
            "squad_ref": draft.squad_transaction_ref})

    return {"status": "filed", "squad_ref": draft.squad_transaction_ref}
```

SQUAD SANDBOX SETUP

Register at sandbox.squadco.com

Generate sandbox API key and webhook secret

Register webhook URL: <https://your-backend.com/api/v1/webhooks/squad>

Add to .env: SQUAD_SECRET_KEY and SQUAD_WEBHOOK_SECRET

Test with Squad's sandbox transaction simulator

Confirm VERA dashboard updates in real time when test transaction fires

05

New Features for VERA

Five new features on top of the existing GRACE foundation. Each is mapped to a specific judging criterion it improves.

Feature 1 — Isolation Forest Anomaly Detection

Adds a trained ML model as a second detection layer alongside the existing heuristic engine. The Isolation Forest learns what 'normal' looks like from the transaction data and flags statistical outliers — including patterns the heuristic rules haven't defined.

Property	Detail
Scoring criterion	AI Technical Depth (30%) — adds a trained model story
Algorithm	scikit-learn IsolationForest
Training data	Synthetic dataset: data/synthetic/transactions.csv (2,000 records)
Features	transaction_amount, hour_of_day, day_of_week, sender_degree, receiver_fan_in, channel_encoded
Output	anomaly_score (-1 = outlier, 1 = normal) + contamination probability
Integration point	Runs in parallel with heuristic detection after graph is built

Owner ML Dev

backend/app/services/anomaly_service.py (NEW FILE)

```
import numpy as np
import joblib
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
import pandas as pd
from pathlib import Path

MODEL_PATH = Path("models/isolation_forest.joblib")
SCALER_PATH = Path("models/scaler.joblib")

FEATURES = [
    "amount_ngn", "hour_of_day", "day_of_week",
    "sender_degree", "receiver_fan_in", "is_round_amount"
]

def train_model(transactions: list[dict]) -> None:
    df = pd.DataFrame(transactions)
    df["hour_of_day"] = pd.to_datetime(df["timestamp"]).dt.hour
    df["day_of_week"] = pd.to_datetime(df["timestamp"]).dt.dayofweek
    df["is_round_amount"] = (df["amount_ngn"] % 1000 == 0).astype(int)
```

```
# sender_degree and receiver_fan_in injected from graph service

X = df[FEATURES].fillna(0).values
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

model = IsolationForest(
    n_estimators=200,
    contamination=0.05, # expect ~5% anomalous
    random_state=42,
    n_jobs=-1
)
model.fit(X_scaled)

MODEL_PATH.parent.mkdir(exist_ok=True)
joblib.dump(model, MODEL_PATH)
joblib.dump(scaler, SCALER_PATH)

def score_transaction(txn: dict, sender_degree: int,
    receiver_fan_in: int) -> dict:
    if not MODEL_PATH.exists():
        return {"anomaly_score": 0, "is_anomaly": False,
            "confidence": 0.0}

    model = joblib.load(MODEL_PATH)
    scaler = joblib.load(SCALER_PATH)

    ts = pd.to_datetime(txn["occurred_at"])
    features = np.array([[
        txn["amount_ngn"],
        ts.hour,
        ts.dayofweek,
        sender_degree,
        receiver_fan_in,
        int(txn["amount_ngn"] % 1000 == 0),
    ]])

    X_scaled = scaler.transform(features)
    score = model.score_samples(X_scaled)[0] # lower = more anomalous
    prediction = model.predict(X_scaled)[0] # -1 = anomaly, 1 = normal

    return {
        "anomaly_score": float(score),
        "is_anomaly": prediction == -1,
        "confidence": float(max(0, min(1, -score / 0.5)))
    }
```

Demo moment: Show the Isolation Forest flagging a transaction the heuristic engine missed. 'Our rules look for known patterns. Our trained model looks for anything statistically unusual — even patterns we haven't defined yet.'

Feature 2 — NLP Entity Name Verification

Adds an NLP layer that flags suspicious entity names and detects identity fragmentation. Shell company indicators, near-identical names used to fragment an entity across the graph, and BVN mismatches are all surfaced as trust verification signals.

Property	Detail
Scoring criterion	AI Technical Depth (30%) — adds NLP to AI stack alongside graph + anomaly detection
Methods	RapidFuzz name similarity, keyword detection (Holdings, Ventures, Enterprise patterns), BVN cross-reference
Output	entity_name_risk_flags: list of flags per entity
Integration point	Runs during entity resolution in the ingest worker
Owner	ML Dev

Feature 3 — Entity Trust Score Card

A visual trust score card on every entity detail page — a clear, interpretable output that directly answers the challenge brief. Shows the trust score, a color-coded verification badge, evidence summary, and a verification status.

Property	Detail
Scoring criterion	Presentation & Demo (15%) + directly answers "clear interpretable output" from the mission brief
Trust levels	VERIFIED (green shield, score < 0.4), FLAGGED (amber shield, 0.4-0.69), HIGH RISK (red shield, >= 0.7)
Card contents	Score badge, pattern detected, evidence count, anomaly flag, name verification status, Squad transaction count
Owner	Frontend Dev

Demo moment: Show the trust score card on the beneficiary entity after the POS ring is detected. The red HIGH RISK badge with supporting evidence is instantly understood by every judge in the room.

Feature 4 — Responsible AI Panel

A dedicated dashboard page covering false positive handling, bias documentation, and privacy controls. Directly targets the bonus impact criterion and separates VERA from every team that ignores responsible AI entirely.

Property	Detail
Scoring criterion	Impact Potential bonus (10%) — responsible handling of bias, privacy, false positives
Sections	False Positive Rate (from audit log), Bias Documentation, Privacy Controls, Threshold Calibration
False positive metric	Computed from audit log: alerts reviewed vs alerts dismissed by compliance officer
Bias statement	Thresholds calibrated for Nigerian transaction patterns, not US/European baselines. POS volume that looks suspicious globally is normal in Lagos.
Privacy controls	PII stripped before LLM calls, data stays in customer infrastructure, NDPA 2023 compliant
Owner	Frontend Dev for UI, ML Dev for metrics

Feature 5 — Squad Transaction Live Feed

A live feed panel on the dashboard showing incoming Squad transactions in real time as they arrive via webhook. Each transaction shows amount, entities, channel, and a risk indicator. This is the visual proof that Squad is genuinely powering the data pipeline.

Property	Detail
Scoring criterion	Squad API Integration (20%) — makes the integration unmissable in the demo
Data source	GET /api/v1/transactions/recent (new endpoint, returns last 20 transactions with Squad source flag)
Refresh	React Query polling every 10 seconds, or WebSocket if time allows
Visual	Transaction cards with Squad logo badge on Squad-sourced transactions, risk color coding
Demo moment	Trigger a Squad sandbox transaction live. Judges watch it appear in the feed and propagate to the graph.
Owner	Frontend Dev

06

Team Roles & Responsibilities

Four people, clear ownership, no overlap. Every feature maps to exactly one owner. The build is parallelizable — all four workstreams can run simultaneously after Day 1 Squad integration is done.

BACKEND / INTEGRATION DEV	ML DEV
- Squad webhook endpoint (POST /api/v1/webhooks/squad) - HMAC signature verification - Squad payload -> TransactionIngestItem mapping - STR filing endpoint (POST /api/v1/str/{id}/file) - Squad payment initiation on filing - Squad transaction reference storage on STR record - New env vars: SQUAD_SECRET_KEY, SQUAD_WEBHOOK_SECRET - Squad sandbox registration and testing - Anomaly service integration into ingest worker - GET /api/v1/transactions/recent endpoint - Add squad_transaction_ref field to STR model - Deploy updates to DigitalOcean	- Isolation Forest model training script - Feature engineering: amount, hour, day, degree, fan-in, round_amount - Model serialization with joblib (models/ directory) - score_transaction() integration with detection service - NLP entity name verification service - Shell company pattern detection (Holdings, Ventures keywords) - Name similarity clustering with RapidFuzz - BVN cross-reference anomaly flagging - Anomaly score field on alert records - False positive rate computation from audit log - Bias documentation for Responsible AI panel - Model performance metrics (precision, recall on synthetic data)
FRONTEND DEV	TEAM LEAD

Entity trust score card component (badge + evidence summary) Squad live transaction feed panel on dashboard Squad logo badge on Squad-sourced transactions Squad transaction reference display on STR detail page Responsible AI panel page (false positive rate, bias, privacy) Before/after demo contrast (raw CSV vs live graph) Trust level badge: VERIFIED / FLAGGED / HIGH RISK Anomaly flag indicator on transaction cards Updated architecture diagram with Squad integration NLP flag display on entity detail page Demo rehearsal and polish One-pager PDF for judges	Pitch deck (10 slides, exact Squad format) User interviews: 2-3 compliance officers or fintech staff Fraud case study: documented Nigerian POS ring case Impact numbers: MFBs, transaction volume, fraud prevented Demo script and rehearsal (4 min 30 sec) Q&A; preparation and cheat sheet GitHub repo documentation cleanup Responsible AI section for pitch deck Team coordination and build sequencing Slide 2 user persona: name, role, real quote Research & validation evidence for Slide 9
---	--

07

Build Sequence & Timeline

Build in dependency order. The Squad webhook must be working before the frontend can show the live feed. The Isolation Forest must be trained before it can be integrated. Everything else is independent.

Priority	Task	Owner	Time	Blocks
P0 — Critical	Squad webhook endpoint + HMAC verification	Backend Dev	2-3 hrs	Everything Squad-related
P0 — Critical	Squad sandbox registration + test transaction	Backend Dev	30 min	Live demo
P1 — High	Squad payment trigger on STR filing	Backend Dev	1-2 hrs	Demo closing moment
P1 — High	Isolation Forest training script + model file	ML Dev	2-3 hrs	Anomaly integration
P1 — High	Anomaly service integration into detection pipeline	ML Dev	1-2 hrs	AI depth score
P1 — High	NLP entity name verification	ML Dev	2-3 hrs	AI depth score
P2 — Medium	Entity trust score card (frontend)	Frontend Dev	2-3 hrs	Demo visual
P2 — Medium	Squad live transaction feed panel	Frontend Dev	2-3 hrs	Squad demo moment
P2 — Medium	Squad reference on STR detail page	Frontend Dev	1 hr	Squad demo moment
P2 — Medium	Responsible AI panel	Frontend Dev + ML Dev	2 hrs	Bonus criterion
P3 — Polish	Before/after demo contrast	Frontend Dev	1 hr	Presentation
P3 — Polish	Architecture diagram updated	Team Lead	1 hr	Pitch deck

P3 — Polish	User interviews (2-3 conversations)	Team Lead	Before event	Domain insight score
P3 — Polish	10-slide pitch deck	Team Lead	3-4 hrs	Presentation
P3 — Polish	One-pager PDF	Team Lead	1 hr	Submission

08

Demo Flow — 5 Minutes

The demo is the highest-leverage 5 minutes of the entire hackathon. Every step is designed to answer a judging criterion visually.

0:00 — The hook (30 sec)

Open with the raw CSV. Show 2,000 rows of transactions. 'This is what a compliance team sees. 2,000 transactions. Somewhere in here is a fraud ring. It would take a human analyst days to find it. Watch what VERA does in 30 seconds.'

0:30 — Squad transaction fires (30 sec)

Trigger a Squad sandbox transaction live. Show the VERA dashboard. The transaction appears in the Squad live feed panel with the Squad logo. 'That transaction just came in through Squad's payment infrastructure. VERA is watching every Squad transaction in real time.'

1:00 — Graph lights up (30 sec)

Navigate to Graph Explorer. Show the network. Nodes colored by trust level. 'This is what the fraud looks like as a network. 8 accounts, all sending to the same beneficiary. Each one looks clean individually. Together — that's a cash-out ring.'

1:30 — Alert and trust score card (45 sec)

Click the POS ring alert. Show the alert detail with subgraph. Click through to the beneficiary entity. Show the trust score card: RED SHIELD, HIGH RISK, 87% confidence, 3 red flags listed. Show the Isolation Forest anomaly flag alongside the heuristic detection. 'Two AI systems independently flagged this entity. Rule-based detection and our trained anomaly model.'

2:15 — STR generation (45 sec)

Type reviewer note. Click Generate STR. Wait for Groq. 'AI drafts the full regulatory report from the graph evidence. What a compliance analyst would spend 4 hours writing — done in 8 seconds.'

3:00 — File via Squad (30 sec)

Click Approve and File. Show the Squad transaction reference appearing on the STR detail page. 'VERA initiates the financial action through Squad. Squad powers the intelligence at the front and the action at the back.'

3:30 — Responsible AI (30 sec)

Navigate to the Responsible AI panel. Show false positive rate, bias documentation, privacy controls. 'We built this responsibly. Every threshold is tuned for Nigerian patterns. No automated filing. Human in the loop on every decision.'

4:00 — Close (30 sec)

'VERA is live. Deployed. The pipeline works on real data. Squad transaction in. Trust score out. STR filed. That is VERA.'

DEMO CHECKLIST — RUN THIS THE NIGHT BEFORE

- Squad sandbox API key working and set in production env
- Test Squad webhook fires and appears in VERA dashboard
- Isolation Forest model trained and serialized in models/
- Trust score card shows on beneficiary entity
- STR filing triggers Squad payment and shows reference
- Responsible AI panel has real false positive numbers
- Full demo rehearsed end-to-end under 5 minutes
- Browser notifications disabled, zoom at 100%, tab pre-loaded
- Mobile hotspot as backup in case venue WiFi fails

09

Projected Score vs Competition

Criterion	Weight	VERA Score	Typical Team Score	VERA Advantage
AI Technical Depth	30%	9/10	5/10 — GPT wrapper, no trained model	Graph + Isolation Forest + LLM + NLP vs one LLM call
Squad API Integration	20%	9/10	6/10 — payment button only	Data source + action layer vs decorative integration
Problem Relevance & Domain Insight	15%	9/10	6/10 — generic fraud framing	Nigerian patterns, NFIU, CBN, real user quotes
Solution Design & Scalability	15%	9/10	4/10 — demo-only architecture	Production deployed, async workers, graph DB, audit trail
Presentation & Demo	15%	9/10	6/10 — screenshots or broken demo	Live deployed system, D3 graph, trust score card
Impact Potential (bonus)	10%	8/10	2/10 — not addressed	Responsible AI panel, false positive rate, bias docs

Weighted score projection: VERA ~91% vs typical competitor ~52%. The gap is widest on AI Technical Depth (the highest-weighted criterion) and Solution Design. These are structural advantages that cannot be closed in 48 hours by a team starting from scratch.

VERA sees what traditional systems can't. The network behind the transaction.

VERA — Built by Team Veltrix for Squad Hackathon 3.0.
Upgraded from GRACE — 3rd place, Antler Lagos x Lua AI Hackathon.